

Computación Paralela y Distribuida

2026-II

José Fiestas PhD

12 de agosto de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe





Al finalizar la unidad, los alumnos conocen:

1. Modelo secuencial (y limitaciones)
2. Paralelismo y arquitecturas paralelas (paradigmas)
3. Modelos escalables (optimización en paralelo)
4. **Taxonomía de Flynn**
5. **Diseño en paralelo: método de Foster**
6. DAG (Directed Acyclic Graphs)

U1.4 Taxonomia de Flynn

Clasificación de arquitecturas de computadoras propuesta por Michael J. Flynn (1966), basada en

- Flujo de instrucciones (Instruction Stream): secuencia de instrucciones ejecutadas.
- Flujo de datos (Data Stream): datos sobre los cuales actúan esas instrucciones.



		Datos	
		Single	Multiple
Instrucciones	Single	SISD	SIMD
	Multiple	MISD	MIMD

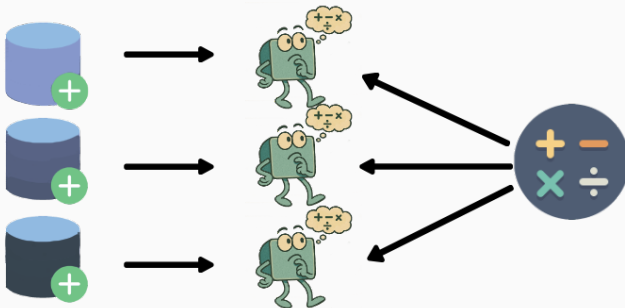
Procesadores secuenciales ejecutan una instrucción sobre una unidad de memoria (Arquitectura Von Neumann es SISD) E.g. microcontroladores

SISD



Una única instrucción (tarea) se ejecuta en uno o más núcleos en múltiples **data streams**, en forma simultánea (GPUs, arquitecturas vectoriales) Algoritmos: suma de vectores, filtros de imagen

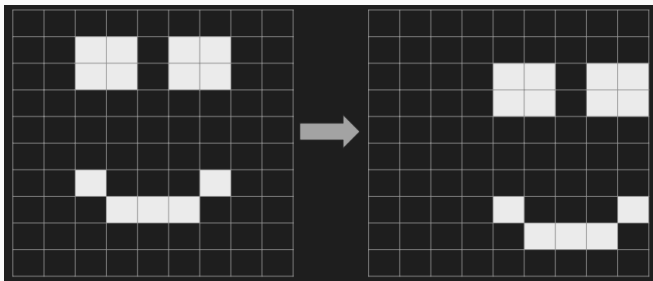
SIMD



E.g. moviendo una imagen pixel a pixel.

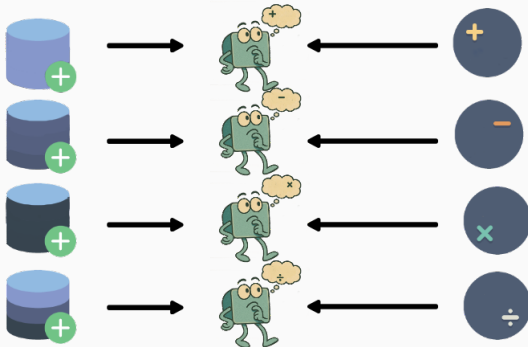
Instrucción:

$x+2, y-1$



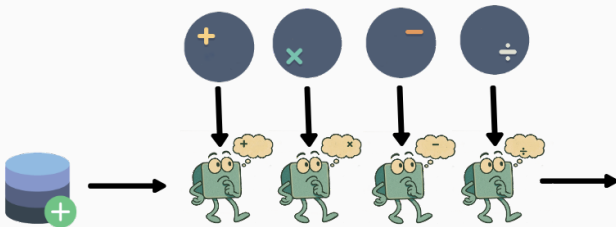
Múltiples tareas en múltiples procesos se ejecutan en distintos **data streams** en forma simultánea (arquitecturas de memoria compartida o distribuída, GPU modernas en modo GPGPU) Algoritmos: multiplicación de matrices con hilos independientes, N-Body paralelo,

MIMD



Múltiples procesos ejecutan distintas instrucciones en la misma data. E.g. sistemas tolerantes a fallos en tiempo real (sistemas de navegación aérea),

MISD



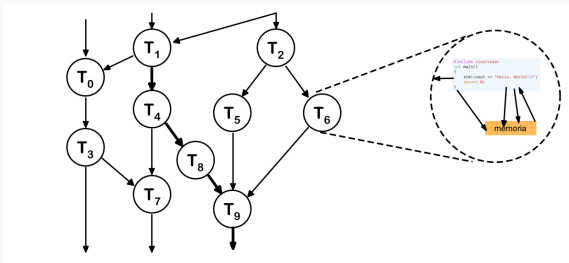
U1.5 Diseño en paralelo: método de Foster



Diseño en paralelo: tasks y channels Método de Foster (1995)

En paralelismo, se utilizan dos abstracciones fundamentales:

- **Task**, programa, memoria local y puertos de entrada/salida.
- **Channel**, mensaje que conecta la salida de un task con la entrada de otro. Mecanismo que define la dependencia entre tasks.





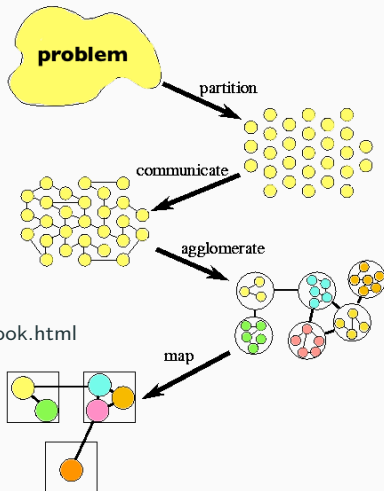
Características:

- Un cálculo en paralelo consiste de dos o más tasks, que se ejecutan en forma concurrente/simultánea
- Cada task es **secuencial** en memoria local, y realiza operaciones de *lectura*, *escritura*, *envío* y *recibo* de datos, *creación* de nuevos tasks, y *término* de los mismos.
- **envío** de datos es **asíncrono**, **recibo** de datos es **síncrono** (causa bloqueo)

Procedimiento de 4 pasos
para el diseño de un
algoritmo en paralelo

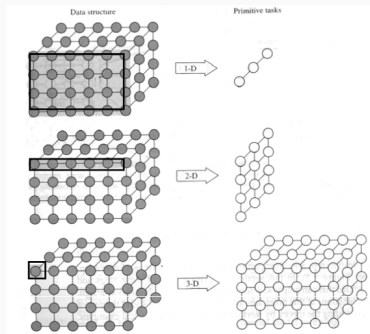
De: Ian Foster, Designing and Building
Parallel Programs, 2003.

<https://www.mcs.anl.gov/~itf/dbpp/text/book.html>



En ambos casos, se trata de encontrar la mayor cantidad de piezas (*primitive tasks*) posible, que brindan un límite superior al paralelismo.

- la cantidad de *primitive tasks* es por lo menos un orden de magnitud mayor que la cantidad de recursos (procesos) utilizados
- *primitive tasks* son aproximadamente del mismo tamaño (particionamiento óptimo)





2. Comunicación, (overhead) se realiza a través de canales abiertos entre tasks, definidos en el paso 1. Se busca que:

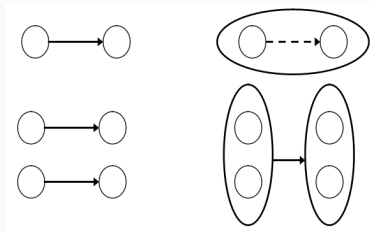
- la comunicación sea balanceada entre tasks
- cada task comunique solo a un número pequeño de vecinos
- tasks puedan comunicar en forma simultánea

La comunicación puede ser:

- **Local**, cuando un task necesita información de un número pequeño de otros tasks para ejecutar una tarea
- **Global**, cuando un número significativo de tasks debe interactuar para completar una tarea

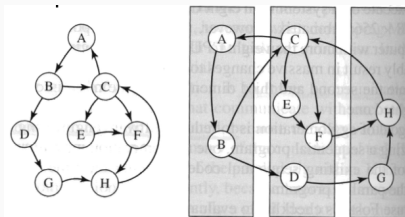
3. Aglomeración, consiste en agrupar tasks con el fin de elevar performance o simplificar la programación. Aglomeración busca:

- reducir el *overhead*
(i.e. agrupar tasks que implican comunicación, agrupar tasks que no pueden ser concurrentes, agrupar tasks que envían y que reciben para reducir la latencia)
- que el diseño garantice escalabilidad, para aprovechar el máximo de recursos (procesos)



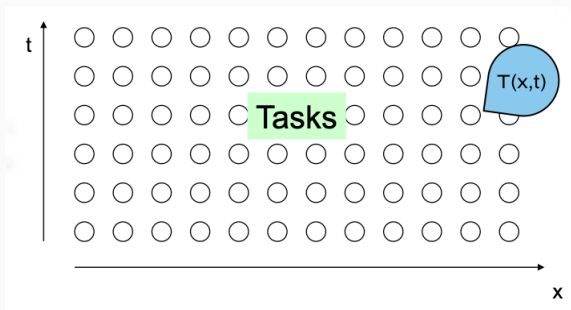
4. Mapping, consiste en asignar tasks a procesos.

- Se busca maximizar la utilización de procesos (cómputo correctamente balanceado) y minimizar la comunicación entre ellos (tasks dependientes se asignan al mismo proceso)
- se considera asignar tanto un task como multitasks por proceso
- se considera la asignación estática y dinámica de tasks a procesos



La tarea (task) consiste en un cálculo por celda que depende de celdas vecinas

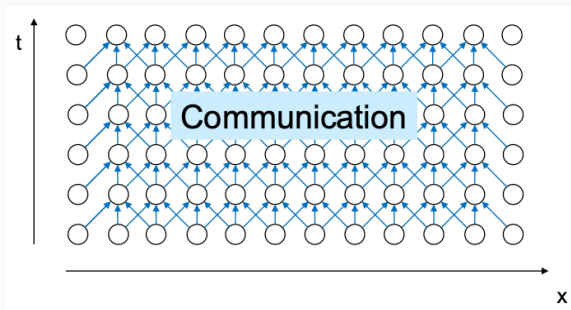
$$T(x, t) = f(T(x, t - 1), T(x - 1, t - 1), T(x + 1, t - 1))$$



El dominio se particiona en tareas primitivas ("primitive tasks")

La tarea (task) consiste en un cálculo por celda que depende de celdas vecinas

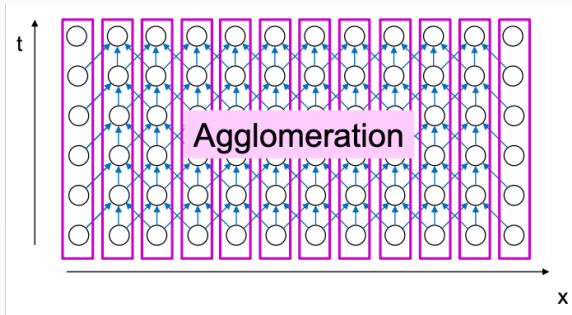
$$T(x, t) = f(T(x, t - 1), T(x - 1, t - 1), T(x + 1, t - 1))$$



Se determina el nivel de comunicación entre tareas ("tasks")

La tarea (task) consiste en un cálculo por celda que depende de celdas vecinas

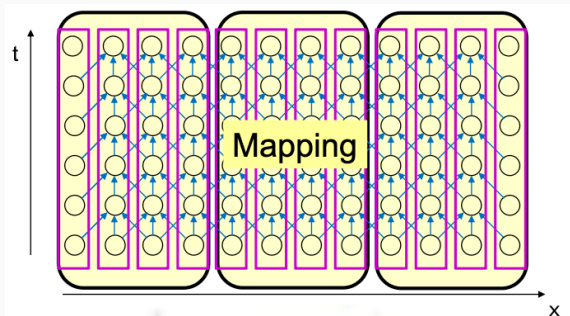
$$T(x, t) = f(T(x, t-1), T(x-1, t-1), T(x+1, t-1))$$



Las tareas ("tasks") se agrupan en tareas más largas para incrementar performance (reducir canales de comunicación)

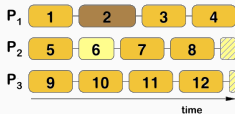
La tarea (task) consiste en un cálculo por celda que depende de celdas vecinas

$$T(x, t) = f(T(x, t - 1), T(x - 1, t - 1), T(x + 1, t - 1))$$

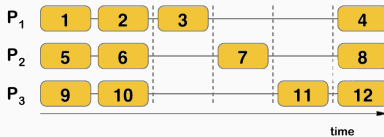


Las tareas ("tasks") se asignan a procesos (descomposición de dominio y balanceo de carga)

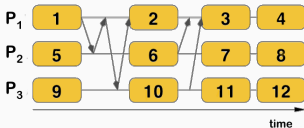
- Desbalance de cargas



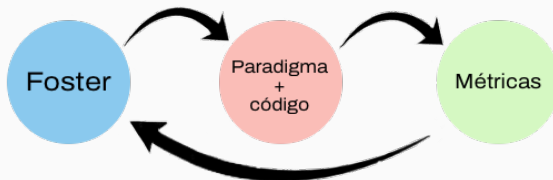
- Cuellos de botella



- Comunicación



- Aplique **Foster** (particionar-comunicar-aglomerar-mapear)
- Escoja el **paradigma** más cercano al problema (compartido, distribuido, híbrido) y desarrolle el algoritmo de acuerdo al paradigma elegido (pseudocódigo-código). Diseñe un plan de ejecución (**scheduling**), de acuerdo a las dependencias y el I/O
- Evalúe **métricas** que muestren el grado de **sincronización** y **comunicación** entre procesos.
- Si el código no es eficiente, revise el primer paso



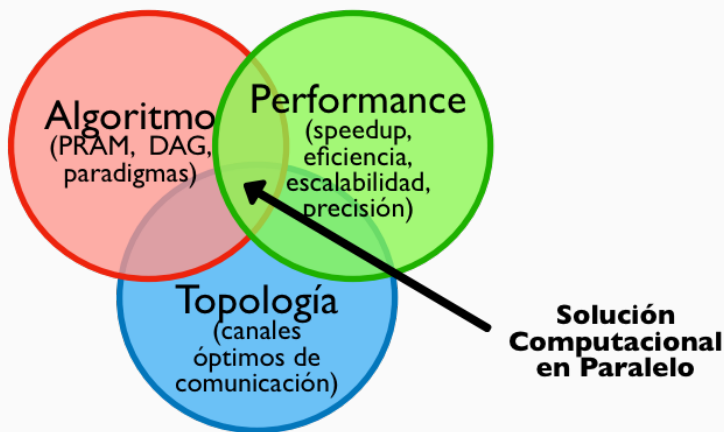


- **Primer objetivo:** *minimizar el tiempo total de ejecución*
 - **Tiempo de cálculo** propiamente dicho, se optimiza asignando tareas a procesadores distintos para maximizar la concurrencia
 - **Tiempo de comunicación/sincronización**, asignando tareas independientes en cada proceso
- **Segundo objetivo:** mantener (o mejorar) la *precisión del algoritmo* secuencial correspondiente. Este no es un problema trivial

Complejidad



Precisión





Describa los pasos necesarios para paralelizar la suma secuencial de n números, utilizando el Método de Foster. Aplique el método para $n=10^{12}$ e indique la cantidad óptima de procesadores necesarios para paralelizar esta tarea en forma eficiente. Desarrolle los 4 pasos de Foster, y determine:

- Particionamiento: el *primitive task*
- Comunicación: la cantidad de canales de comunicación (inicial)
- Aglomeración: la cantidad final de *channels* (describa el criterio de aglomeración)
- Mapeo: cada proceso puede sumar como máximo 10^9 números

Si la suma de dos números se realiza en un ciclo de 10^{-9} segundos, y el envío/recibo de datos sucede con un ancho de banda de 10^{-8} segundos / Byte, calcule el tiempo total de ejecución $T_{pej} = T_{comp} + T_{comm}$ ¿Es el tiempo en paralelo menor que el secuencial?



-  Ian Foster, *Designing and Building Parallel Programs*
<https://www.mcs.anl.gov/~itf/dbpp/text/book.html>
-  Peter S. Pacheco, *An Introduction to Parallel Programming*. 1st.
Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5.
-  Michael J. Quinn, *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn:
0071232656.

